

CleanSpace Development Context

This file is the persistent engineering context for the CleanSpace project.

The AI CLI must read this file **before making changes** and update it **after completing a meaningful change, bug fix, refactor, or milestone**.

The purpose of this file is to make development:

- easier to debug
 - easier to resume after a break
 - easier for another AI/session/developer to understand
 - safer when modifying filesystem code
 - easier to track architectural decisions
 - easier to avoid repeating previous mistakes
 - professional and auditable
-

1. Project Overview

Project Name

CleanSpace

Product Type

Local-first Linux desktop application.

Main Purpose

CleanSpace helps users understand, organize, and safely clean files on their computer.

Core product goals:

- scan folders
- categorize files
- analyze disk usage
- organize files safely
- preview changes before applying
- undo file organization operations
- detect duplicate files
- identify cleanup candidates
- generate plans such as:

I need 10 GB free.

- send unwanted files to Linux Trash instead of permanently deleting them
- monitor selected folders for new files
- remain private and local-first

2. Technology Stack

Current intended stack:

```
Language:
TypeScript

Desktop:
Electron

UI:
React

Frontend Build Tool:
Vite

Runtime:
Node.js

Database:
SQLite

Platform:
Linux

Architecture:
Electron Main Process
    ↓
Secure IPC
    ↓
Preload API
    ↓
React Renderer
```

Do not change the major stack without documenting the reason in the Architecture Decisions section.

3. Development Principles

The project should prioritize:

1. filesystem safety
2. predictable behavior
3. user trust
4. maintainable architecture
5. testability
6. performance
7. polished UX
8. portfolio-quality engineering

Never sacrifice filesystem safety for convenience.

4. AI CLI Operating Rules

Before doing any development work, the AI must:

1. read this entire `context.md`
2. inspect `git status`
3. inspect the relevant source files
4. inspect recent changes when necessary
5. determine the current milestone
6. understand existing tests before modifying related functionality

Do not assume the repository matches a previous conversation.

The repository is the source of truth.

5. Change Tracking Rule

After every meaningful implementation, update:

- Current Status
- Completed Features
- Known Issues
- Recent Changes
- Files Changed
- Tests
- Architecture Decisions if applicable
- Next Recommended Task

Do not rewrite the entire history unnecessarily.

Append new development entries to the Development Log.

6. Current Status

Current Milestone

Milestone 0 — Foundation

Current State

Not yet confirmed from repository inspection.

The CLI must update this section after inspecting the project.

Last Known Working State

Unknown until verified.

Do not claim the application works unless it has been tested.

7. Milestone Roadmap

Milestone 0 — Foundation

Target:

- Electron
- React
- Vite
- TypeScript
- strict TypeScript configuration
- ESLint
- Prettier
- test framework
- secure preload
- typed IPC foundation
- application shell

Definition of done:

- application starts successfully on Linux
- renderer loads
- secure preload works
- lint passes
- TypeScript passes
- tests pass
- production build succeeds where possible

Status:

NOT STARTED / VERIFY

Milestone 1 — Folder Selection

Target:

Allow user to select a folder through a native dialog.

Status:

NOT STARTED

Milestone 2 — Scanner

Target:

Safely recursively scan a selected folder.

Collect:

- path
- filename
- extension
- size
- timestamps
- parent folder
- initial category information

Status:

NOT STARTED

Milestone 3 — File Browser

Target:

Display scanned files.

Features:

- sorting
- filtering
- file size
- type
- category
- modified date

Status:

NOT STARTED

Milestone 4 — Categorization

Target:

Deterministic file categorization.

Status:

NOT STARTED

Milestone 5 — Storage Analytics

Target:

Show:

- total scanned storage
- category totals
- largest files
- largest folders

Status:

NOT STARTED

Milestone 6 — Organization Preview

Target:

Generate proposed file moves without modifying files.

Status:

NOT STARTED

Milestone 7 — Apply + Undo

Target:

Safely execute organization operations and support undo.

Status:

NOT STARTED

Milestone 8 — Duplicate Detection

Target:

Efficient duplicate discovery.

Status:

NOT STARTED

Milestone 9 — Cleanup Analysis

Target:

Generate cleanup candidates with reasons and risk levels.

Status:

NOT STARTED

Milestone 10 — Free X GB Planner

Target:

Support requests such as:

I need 10 GB free.

Status:

NOT STARTED

Milestone 11 — Linux Trash

Target:

Move selected unwanted files to Linux Trash safely.

Status:

NOT STARTED

Milestone 12 — Folder Monitoring

Target:

Watch selected folders such as Downloads.

Status:

NOT STARTED

Milestone 13 — UX Polish

Target:

- progress
- cancellation
- notifications
- loading states
- empty states

- keyboard accessibility
- themes

Status:

NOT STARTED

Milestone 14 — Performance

Target:

Benchmark and improve real bottlenecks.

Status:

NOT STARTED

Milestone 15 — Security Audit

Target:

Full filesystem and Electron security review.

Status:

NOT STARTED

Milestone 16 — Packaging

Target:

Linux distributables.

Potential formats:

- AppImage
- `.deb`
- Flatpak later

Status:

NOT STARTED

Milestone 17 — Portfolio Release

Target:

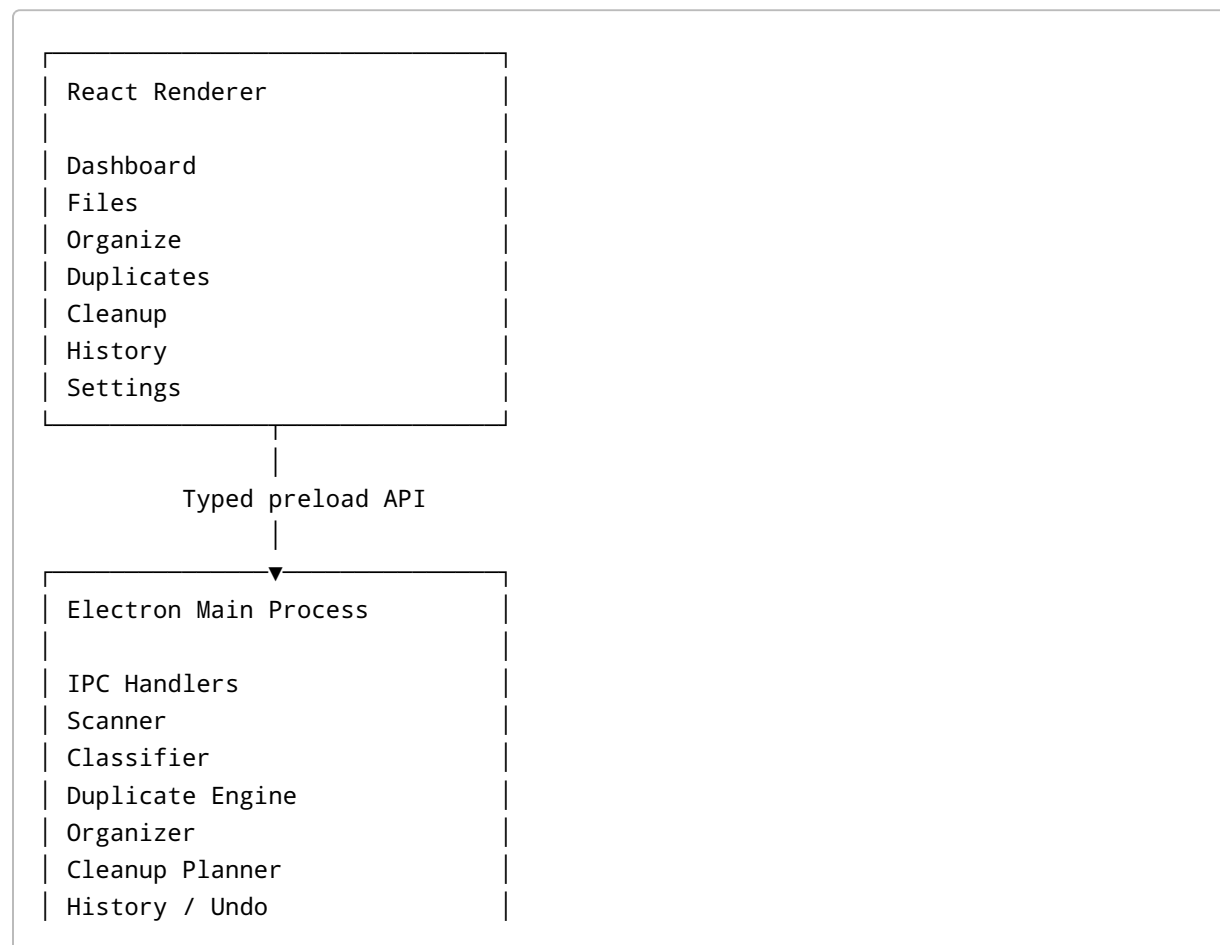
- excellent README
- screenshots
- demo
- architecture documentation
- release notes

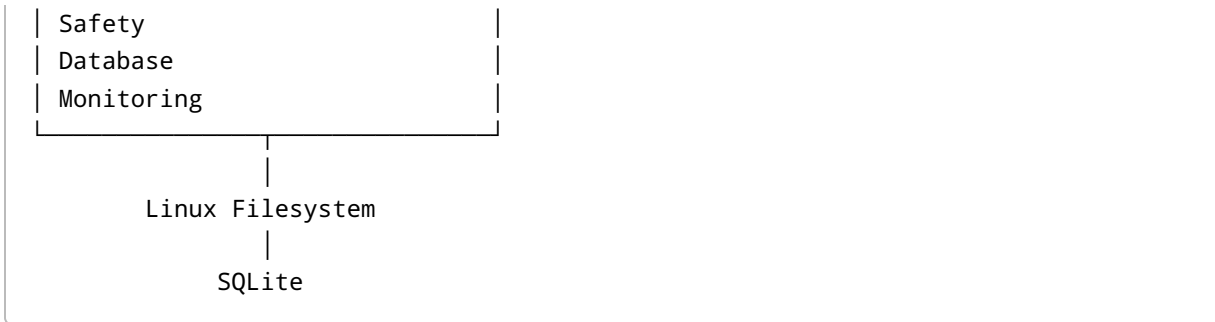
Status:

NOT STARTED

8. Architecture

Expected high-level architecture:





9. Intended Source Structure

The exact repository must be inspected before assuming this exists.

Preferred structure:

```
src/
├── main/
│   ├── ipc/
│   ├── scanner/
│   ├── classifier/
│   ├── duplicates/
│   ├── organizer/
│   ├── cleanup/
│   ├── history/
│   ├── database/
│   ├── safety/
│   └── monitoring/
├── preload/
├── renderer/
│   ├── components/
│   ├── pages/
│   ├── hooks/
│   ├── stores/
│   ├── lib/
│   └── types/
└── shared/
    ├── types/
    ├── schemas/
    └── constants/
```

Tests should mirror source responsibilities when reasonable.

10. Electron Security Requirements

These are non-negotiable unless explicitly reconsidered and documented.

Use:

```
contextIsolation = true  
nodeIntegration = false
```

Renderer code must not receive unrestricted Node.js access.

Only expose narrowly scoped functions through preload.

Bad:

```
window.api.runAnything(command)
```

Good:

```
window.cleanSpace.selectFolder()  
window.cleanSpace.scanFolder(path)  
window.cleanSpace.getScanSummary(scanId)
```

IPC arguments must be validated.

Never trust paths supplied by the renderer.

11. Filesystem Safety Rules

The filesystem layer is security-sensitive.

Never automatically scan or modify protected Linux paths such as:

```
/proc  
/sys  
/dev
```

```
/run  
/boot  
/bin  
/sbin  
/lib  
/usr  
/etc  
/root
```

Some directories may eventually be readable for advanced features, but never enable this silently.

Default user-facing scan targets should include:

```
~/Downloads  
~/Desktop  
~/Documents  
~/Pictures  
~/Videos
```

Do not recursively follow symbolic links by default.

Detect symlinks explicitly.

Do not overwrite an existing file silently.

Before moving files:

1. validate source path
2. validate destination path
3. check allowed roots
4. check for destination collision
5. check whether source still exists
6. record intended operation
7. perform operation
8. verify result
9. update history

12. Destructive Action Policy

Permanent deletion should not be the default.

Preferred behavior:

```
Remove
  ↓
Linux Trash
```

Any operation that changes files must support a preview.

Examples:

- move
- rename
- organize
- cleanup
- duplicate removal

The UI must clearly show what will happen.

13. Undo Requirements

Organization actions must be reversible when possible.

Every file operation should have enough metadata to reconstruct the previous state.

Example:

```
operation:
MOVE

source:
/home/user/Downloads/report.pdf

destination:
/home/user/Downloads/Documents/report.pdf
```

Undo:

```
/home/user/Downloads/Documents/report.pdf
→
/home/user/Downloads/report.pdf
```

Undo must verify that it will not overwrite a newly created file.

14. Database Responsibilities

SQLite should store durable application state.

Expected entities:

```
scans
files
operations
operation_items
settings
organization_rules
```

Do not spread SQL queries randomly throughout the project.

Use repositories or a small dedicated data-access layer.

Use schema migrations.

15. File Scanner Requirements

Scanner must eventually handle:

- recursive directories
- empty directories
- inaccessible directories
- permission errors
- disappearing files
- Unicode filenames
- spaces
- symbolic links
- large files
- large folder trees
- cancellation
- progress

Avoid synchronous filesystem APIs for long-running scans.

Do not load file contents when only metadata is needed.

16. Categorization Strategy

MVP categorization is deterministic.

Base categories:

```
Documents
Images
Videos
Audio
Archives
Installers
Code
Disk Images
Other
```

Classification inputs may include:

- extension
- MIME type
- filename
- parent folder
- size
- dates

Do not add AI before deterministic categorization is reliable.

17. Duplicate Detection Strategy

Do not immediately hash every file.

Preferred process:

```
All files
↓
Group by size
↓
Groups with 2+ files
↓
Partial hash
↓
Possible matches
```


↓
Full hash
↓
Confirmed duplicates

Do not declare files duplicates from size alone.

Do not rely only on partial hashes.

18. Cleanup Candidate Model

Potential cleanup candidate shape:

```
type CleanupRisk = 'safe' | 'review' | 'important';

interface CleanupCandidate {
  path: string;
  sizeBytes: number;
  reason: string;
  risk: CleanupRisk;
  category: string;
}
```

Examples:

```
confirmed duplicate
→ Safe

old installer
→ Safe or Review depending on rules

large personal video
→ Review

unknown system-related file
→ Important / exclude
```

Do not generate cleanup recommendations without a reason.

19. “I Need X GB” Planner

The cleanup planner should accept a target number of bytes.

Example:

```
Target:
10 GB

Candidates:
Duplicate files      4.2 GB
Old installers      2.8 GB
Old archives        1.9 GB
Large unused files  4.1 GB
```

Planner should attempt to satisfy the target while preferring safer candidates.

Conceptual priority:

```
Safe
↓
Review
↓
Important excluded by default
```

The planner should never hide the selected files from the user.

20. Performance Guidelines

Expected target:

Tens of thousands of files should remain usable.

Avoid:

- synchronous recursive scanning
- hashing everything
- giant arrays repeatedly copied in renderer
- sending huge IPC payloads constantly
- updating UI for each discovered file individually

Prefer:

- batches
- pagination or virtualization
- streaming
- async filesystem calls
- worker threads if benchmarks justify them
- incremental progress events
- cancellation

Measure before optimizing.

21. Error Handling

Never silently ignore failures.

Bad:

```
try {  
  await moveFile();  
} catch {}
```

Good:

- capture technical error
- translate it to a meaningful application error
- provide user-safe explanation
- log enough information for debugging

Example:

```
Technical:  
EACCES
```

```
User:  
CleanSpace does not have permission to read this folder.
```

22. Logging Strategy

Use structured logging.

Logs should be useful for debugging but should avoid unnecessary exposure of private user information.

Potential levels:

```
debug
info
warn
error
```

Useful log metadata:

```
operation
module
duration
fileCount
scanId
operationId
errorCode
```

Avoid logging full file paths unnecessarily in production.

During development, richer logging may be enabled.

23. Debugging Procedure

When a bug is reported:

1. reproduce it
2. document reproduction steps
3. inspect relevant logs
4. identify whether the problem is renderer, IPC, main process, filesystem, or database
5. write a failing test when practical
6. implement the smallest fix
7. run related tests
8. run full checks if the fix affects shared code
9. manually verify
10. record the bug and fix in this file

Do not make unrelated refactors during a focused bug fix unless required.

24. Bug Tracking

Add active bugs here.

Format:

BUG-001

Title:

Short descriptive title

Status:

OPEN

Severity:

Low / Medium / High / Critical

Observed:

What happened

Expected:

What should happen

Reproduction:

- 1.
- 2.
- 3.

Suspected Area:

scanner / renderer / IPC / database / etc.

Fix:

Not fixed yet.

Tests:

Not added yet.

Active Bugs

None documented yet.

25. Known Limitations

Keep this section current.

Initial expected limitations:

- Linux only
 - no cloud synchronization
 - no remote account system
 - semantic/AI categorization not part of initial MVP
 - no permanent deletion by default
 - advanced system-folder scanning not supported
-

26. Testing Strategy

Testing is required for core filesystem logic.

Expected categories:

```
Unit tests
Integration tests
Electron E2E tests
Manual acceptance tests
```

Use generated temporary directories.

Never let automated tests reorganize or delete actual user files.

27. Test Commands

The CLI must inspect `package.json` and replace this section with real commands after setup.

Expected examples:

```
npm run lint
npm run typecheck
npm test
```

```
npm run test:e2e
npm run build
```

Current verified commands:

NOT YET VERIFIED

28. Manual Test Checklist

Maintain this as features are implemented.

Foundation

- ☐ App launches on Linux
- ☐ React UI renders
- ☐ DevTools show no unexpected errors
- ☐ Node APIs are inaccessible directly from renderer
- ☐ preload API exists
- ☐ production build completes

Folder Selection

- ☐ Can open folder picker
- ☐ Cancel works
- ☐ Selected path displays correctly

Scanner

- ☐ Can scan empty folder
- ☐ Can scan nested folder
- ☐ Permission failure does not crash app
- ☐ Unicode paths work
- ☐ Symlink behavior is correct

Add new checks as features are implemented.

29. Architecture Decisions

Record decisions using this format:

ADR-001

Date:
YYYY-MM-DD

Decision:
What was decided.

Reason:
Why.

Alternatives:
What other approaches were considered.

Tradeoffs:
What we gain and lose.

Status:
Accepted / Superseded

ADR-001

Date: 2026-08-19

Decision: Use TypeScript + Electron + React for the first production implementation.

Reason: The project needs a polished desktop UI, filesystem access, fast development, and a single primary language.

Alternatives:

- Rust + Tauri
- Java + JavaFX
- C + GTK
- Python + Qt

Tradeoffs:

Electron has a larger runtime and memory footprint than native Rust/C applications, but development velocity and UI ecosystem are stronger for the current project goals.

Status: Accepted

30. Recent Changes

Keep only a useful recent summary here.

Detailed history belongs in Development Log.

Current:

No repository changes documented yet.

31. Files Changed

Update after each development task.

Format:

```
Added:
- src/main/example.ts

Modified:
- src/main/index.ts

Deleted:
- none
```

Current:

No changes recorded.

32. Completed Features

Current:

None verified.

Do not mark something complete based only on code existing.

A feature is complete only after relevant verification.

33. In-Progress Work

Current:

Milestone 0 – Foundation

The repository must be inspected to determine exact progress.

34. Known Technical Debt

Current:

None documented yet.

Add technical debt instead of silently ignoring poor but intentionally temporary solutions.

Example:

TECH-DEBT-001

Area:
Scanner

Problem:
Scanner currently sends one IPC progress event per file.

Reason temporary:
Implemented for MVP simplicity.

Future:
Batch updates every 100 files or 100 ms.

35. Dependency Tracking

When adding an important dependency, document:

Package:
package-name

Purpose:
Why it exists.

Used By:
Feature/module.

Reason Selected:
Why this package rather than alternatives.

Risk:
Maintenance/security/platform concerns.

Do not add large dependencies for trivial tasks.

36. Security Review Checklist

Before major releases verify:

- ☐ nodeIntegration disabled
- ☐ contextIsolation enabled
- ☐ preload exposes minimum necessary functionality
- ☐ IPC input validation exists
- ☐ filesystem paths are validated
- ☐ protected paths are handled
- ☐ symlink behavior is safe
- ☐ move operations prevent overwrites
- ☐ cleanup uses Trash
- ☐ SQL input is parameterized
- ☐ renderer output is safe
- ☐ no arbitrary command execution exists
- ☐ dependencies audited
- ☐ sensitive logs reviewed

37. Definition of Done

A task is only complete when applicable items pass:

- implementation complete
- TypeScript passes

- lint passes
- tests pass
- build passes
- error handling exists
- edge cases considered
- manual test steps documented
- `context.md` updated

Do not mark work finished while known relevant checks are failing.

38. Development Log

Append entries. Do not delete historical entries unless they are clearly incorrect.

Use this exact style.

DEV-000

Date: 2026-08-19

Type: Project initialization context

Milestone: Milestone 0

Summary: Created the persistent development context specification for CleanSpace.

Reason: Future AI CLI sessions need a consistent source of project state, architecture, safety rules, debugging history, and feature progress.

Changes:

- established project goals
- documented technology stack
- documented milestone roadmap
- documented filesystem safety rules
- established bug tracking process
- established architecture decision records
- established change tracking process
- established testing and debugging expectations

Verification: Documentation only.

Next: Inspect the actual repository and begin or resume Milestone 0.

39. Template for Future Development Entries

Copy this section for every meaningful development change.

DEV-XXX

Date:

YYYY-MM-DD

Type:

Feature / Fix / Refactor / Test / Security / Performance / Documentation

Milestone:

Milestone X

Goal:

What we wanted to accomplish.

Before:

Relevant previous state.

Implementation:

What was changed.

Files Changed:

- path
- path

Important Decisions:

Any implementation decisions that future developers need to understand.

Testing:

Commands executed and their results.

Manual Verification:

What was manually tested.

Problems Encountered:

Any failures or unexpected behavior.

Resolution:

How problems were fixed.

Remaining Issues:

Anything intentionally left unresolved.

Commit:
Suggested or actual commit hash/message.

Next:
Recommended next action.

40. Session Handoff

Before stopping a development session, the AI CLI must update this section.

Last Session

Date:

2026-08-19

Worked On:

context.md development workflow definition

Last Verified State:

Repository implementation has not yet been inspected through this context file.

Current Blocker:

None known.

Resume From:

Read context.md, inspect repository, git status, package.json, configs and current Milestone 0 implementation.

Next Recommended Action:

Begin Milestone 0 repository inspection and foundation setup.

41. AI CLI End-of-Task Requirement

At the end of every meaningful task, output a concise summary containing:

```
Completed:
What changed.

Verified:
What tests/checks passed.

Failed:
Anything that did not pass.

Context Updated:
Yes / No

Suggested Commit:
type(scope): description

Next:
Single recommended next task.
```

Then update this file before ending the session.

42. Critical Reminder

This application modifies users' files.

A bug in a normal application may produce a broken screen.

A bug in CleanSpace may cause data loss.

Therefore:

Correctness and recoverability are more important than cleverness.

When uncertain about a file operation:

do nothing and ask the user to review it.